Square-Root Unscented Kalman Filter Bring-Up Guide

Jitterware, Inc.

February 2026

1 Quaternion State Vector

Our state vector $x \in \mathbb{R}^7$ represents the drone's orientation and gyro biases in the (x, y, z, w) order:

$$x = \begin{bmatrix} q_x & q_y & q_z & q_w & b_x & b_y & b_z \end{bmatrix}^T \tag{1}$$

2 Accelerometer Measurement Model

The predicted gravity vector $\hat{z}$ is calculated by projecting the world gravity $g = [0, 0, 1]^T$ into the body frame using the Hamiltonian quaternion q:

$$\hat{z} = \begin{bmatrix} 2(q_x q_z - q_w q_y) \\ 2(q_w q_x + q_y q_z) \\ q_w^2 - q_x^2 - q_y^2 + q_z^2 \end{bmatrix} \tag{2}$$

3 The Innovation Equation

To kill the 3-degree offset, the innovation $\tilde{y}$ must satisfy the condition where measurement noise R is minimized:

$$\tilde{y} = \frac{a_{raw}}{\|a_{raw}\|} - \hat{z} \tag{3}$$

The Kalman Gain K that corrects the state is:

$$K \approx P_{xz}(P_{zz} + R)^{-1} \tag{4}$$

4 Observability and Yaw Drift

In a 6-axis IMU configuration (Accelerometer and Gyroscope), only the Pitch (θ) and Roll (ϕ) are observable through the gravity vector. The Yaw (ψ) represents

a rotation about the gravity vector $g = [0, 0, 1]^T$, making it mathematically unobservable.

$$\dot{\psi} = \omega_z + \text{bias}_{\omega z} \tag{5}$$

Without a secondary reference (Magnetometer or Vision), the Yaw state will integrate the gyroscope bias $\text{bias}_{\omega z}$ over time, leading to a linear drift:

$$\psi(t) = \psi_0 + \int_0^t \epsilon_\omega \, dt \tag{6}$$

where ϵ_ω represents the residual bias after UKF estimation.

5 SR-UKF Core Implementation Reference

5.1 Initialization: SR_UKF_Init

The initialization sets the scaling parameters and weights. The parameter $\alpha = 0.1$ was selected to ensure numerical stability and prevent weight explosion.

$$\lambda = \alpha^2(L + \kappa) - L, \quad W_m^{(0)} = \frac{\lambda}{L + \lambda}, \quad W_c^{(0)} = W_m^{(0)} + (1 - \alpha^2 + \beta) \tag{7}$$

5.2 Sigma Point Generation: GenerateSigmaPoints

Generates $2L + 1$ sigma points using the Cholesky factor S.

$$\mathcal{X}_0 = \bar{x}, \quad \mathcal{X}_i = \bar{x} + \sqrt{L + \lambda} S_i, \quad \mathcal{X}_{i+L} = \bar{x} - \sqrt{L + \lambda} S_i \tag{8}$$

5.3 Prediction: ConstantAngularVelocityModel

Propagates the state using an exponential map (small-angle approximation) to maintain the unit norm of the quaternion q over time interval Δt.

$$q_{k+1} = q_k \otimes \exp\left(\frac{\omega - b_\omega}{2} \Delta t\right) \tag{9}$$

5.4 Measurement: AccelMeasurementModel

Projects the gravity vector into the body frame using the (x, y, z, w) Hamiltonian quaternion mapping:

$$\hat{z} = \begin{bmatrix} 2(xz - wy) \\ 2(yz + wx) \\ w^2 - x^2 - y^2 + z^2 \end{bmatrix} \tag{10}$$

5.5 Square-Root Update: NF_Get_SR_Update & NF_hhvec

Performs the QR decomposition of the compound matrix A to update the Cholesky factor S.

- **NF_hhvec:** Calculates the Householder vector v for a column, with an epsilon-guard (10^{-12}) to prevent division by zero.
- **NF_Get_SR_Update:** Applies Householder transformations to reduce A to an upper triangular matrix R.

5.6 Linear Algebra: Solve_K_BackSub & Solve_Kalman_Gain

Solves the Kalman Gain K without matrix inversion. Since S_z is upper triangular:

$$K(S_z S_z^T) = P_{xz} \implies \text{Step 1: } YS_z^T = P_{xz}, \quad \text{Step 2: } KS_z = Y \tag{11}$$

5.7 Covariance Maintenance: Rank1_Update & Apply_Covariance_Update

Updates the square-root covariance S using sequential Rank-1 Cholesky downdates for each measurement dimension.

$$S_{new} = \text{cholupdate}(S, K \cdot S_{z,i}, -1) \tag{12}$$

5.8 Normalization: NormalizeQuaternion

Enforces the unit constraint on the state quaternion after every addition or integration to prevent magnitude drift.

$$q_{unit} = \frac{q}{\|q\|} \tag{13}$$

6 Numerical Stability and Parameter Tuning

6.1 Sigma Point Weight Scaling (α)

The primary cause of initial filter instability was a weight explosion ($W_m \approx 7 \times 10^4$). For state dimension $L = 7$, an overly small α (10^{-3}) creates a near-singular denominator in the weight calculation:

$$W_m^{(i)} = \frac{1}{2(\alpha^2 L)} \tag{14}$$

By increasing α to 0.1, the denominator was stabilized, reducing the weights to a manageable range (≈ 0.07) and preventing floating-point overflow during the weighted summation of $\hat{z}$.

6.2 Householder QR Epsilon Guard

To handle "Rank Deficient" matrices (occurring when the drone is stationary and process noise is low), a numerical epsilon guard was implemented in `NF_hhvec`:

$$\text{if } \|v\| < 10^{-12} \implies \beta = 0 \tag{15}$$

This prevents the calculation of inf values during the Householder reflection, ensuring the Cholesky factor S remains finite.

6.3 Measurement Noise (R) and Static Offset

A static pitch offset of $1° - 3°$ was observed when $R_{accel} \geq 2.0$. This "Deadband" effect occurs when the measurement update is too weak to overcome gyro bias residuals. Reducing R_{accel} to 0.1 increased the Kalman Gain K, allowing the accelerometer to successfully "pull" the state back to the gravity vector.

6.4 Quaternion Order and Measurement Mapping

The system utilizes an (x, y, z, w) state order. Failure to map the scalar component w to index 3 in the `AccelMeasurementModel` resulted in a 180-degree phase shift in predicted gravity, causing high-frequency oscillations. Correct mapping is:

$$\hat{z}_z = w^2 - x^2 - y^2 + z^2 \tag{16}$$

7 Uncertainty Visualization (Sigma Cloud)

To visualize the 3D covariance ellipsoid in Unity without exceeding serial bandwidth, a round-robin transmission strategy is used. Each telemetry packet contains the mean orientation q_μ and a single sigma point $\mathcal{X}_i$:

$$\text{Packet}_k = \{q_\mu, \mathcal{X}_{k \pmod{15}}, k \pmod{15}\} \tag{17}$$

In the Unity environment, the delta rotation Δq and displacement d for the i-th ghost drone are amplified by a gain factor γ_{vis} to make the mathematical variance visible:

$$q_{ghost,i} = \text{AngleAxis}(\theta_\Delta \cdot \gamma_{vis}, \hat{u}_\Delta) \otimes q_\mu \tag{18}$$

8 Covariance Dynamics

The cloud volume is a visual representation of the Cholesky factor S. The dynamics are governed by:

$$S_{predict} = \text{qr}([W_c(\mathcal{X} - \bar{x}); Q_s]) \tag{19}$$

If the cloud contracts during high-acceleration maneuvers, it implies that the measurement update $S = \text{cholupdate}(S, KS_z, -1)$ is over-correcting, or Q_s is insufficiently capturing the integration noise.

9 Covariance Stability and Process Noise

9.1 Covariance Collapse Phenomenon

If the uncertainty cloud in the visualization environment contracts toward the mean state $\bar{x}$ during motion, it indicates a mathematical collapse of the Cholesky factor S. This occurs when the information gain from measurements consistently outweighs the uncertainty growth from the prediction step:

$$S_k \approx S_{k|k-1} - KS_z \to 0 \tag{20}$$

As $S \to 0$, the generated sigma points $\mathcal{X}_i = \bar{x} \pm \gamma S_i$ become identical to the mean, effectively disabling the Unscented Transformation.

9.2 The Role of Process Noise Q_s

To prevent collapse, the SR-UKF incorporates the Cholesky factor of the process noise Q_s into the augmented A_{pred} matrix before the QR decomposition:

$$A_{pred} = \begin{bmatrix} \sqrt{W_c}(\mathcal{X}_{1..2L} - \bar{x}) \\ Q_s \end{bmatrix} \tag{21}$$

Where Q_s represents the standard deviation of the integration error (e.g., 0.005 rad/s for orientation). This ensures a lower bound on the uncertainty "bloom," allowing the sigma cloud to expand during high-dynamic maneuvers.

9.3 Numerical Guard for S

To maintain a positive-definite covariance in a fixed-point or low-precision environment, a diagonal floor is enforced during the prediction step:

$$S_{ii} = \max(S_{ii}, \epsilon_{min}) \tag{22}$$

where $\epsilon_{min} = 10^{-4}$. This prevents the filter from becoming "infinitely confident" and ignoring subsequent sensor updates.

10 Geometric Distribution of Sigma Points

In a 7-dimensional SR-UKF state vector consisting of 4 orientation components (quaternion) and 3 bias components, the $2L + 1$ (15) sigma points exhibit a specific geometric structure during visualization:

10.1 The 6-Point Rotation Ellipsoid

The first 6 spread points ($\mathcal{X}_{1..6}$) correspond to the orientation uncertainty. In a stabilized state, these points represent the principal axes of rotation error:

$$\mathcal{X}_{rot} = \bar{q} \otimes \exp\left(\pm\gamma S_i\right), \quad i \in \{0, 1, 2\} \tag{23}$$

When the uncertainty is isotropic (equal in all directions), these points appear as a "Sigma Cube," with one pair of points corresponding to each axis:

- **Pitch Pairs:** Displaced along the local X axis.
- **Roll Pairs:** Displaced along the local Y axis.
- **Yaw Pairs:** Displaced along the local Z axis.

10.2 The Central Bias Cluster

The remaining 8 points (the mean $\mathcal{X}_0$ and the bias spread points $\mathcal{X}_{7..14}$) typically remain clumped at the drone's center of mass. This occurs because the `AccelMeasurementModel` provides high-confidence updates for tilt, causing the covariance S for these states to remain small relative to the orientation amplification gain γ_{vis}.

10.3 The Dynamics of the Cloud

The distance d from the mean state $\bar{x}$ to a sigma point $\mathcal{X}_i$ in the Unity environment is a direct visualization of the filter's confidence:

$$d \propto \sqrt{\text{Trace}(S_{predict}) - \text{Trace}(KS_z)} \tag{24}$$

During high-dynamic maneuvers (shaking), the process noise Q_s dominates, causing the cloud to "bloom." When stationary, the measurement update R_s dominates, causing the points to contract back toward the "hover" positions.

11 Standard UKF Baseline Comparison

The traditional Unscented Kalman Filter (UKF) maintains the full state covariance matrix $P \in \mathbb{R}^{7\times7}$ rather than the Cholesky factor S.

11.1 State Covariance Update (Traditional)

In the standard UKF, the covariance is updated via the direct summation of outer products:

$$P_k = \sum_{i=0}^{2L} W_c^{(i)} (\mathcal{X}_i - \bar{x})(\mathcal{X}_i - \bar{x})^T + Q \tag{25}$$

Unlike the SR-UKF, which uses a stable QR decomposition, the standard UKF is susceptible to P becoming non-positive definite due to floating-point truncation, leading to filter divergence.

11.2 Kalman Gain (Traditional)

The Kalman gain in the standard version requires a direct matrix inversion of the innovation covariance S_{zz}:

$$K = P_{xz}(S_{zz})^{-1} \tag{26}$$

In contrast, our SR-UKF utilizes back-substitution on the triangular factor S_z, which is computationally more efficient and numerically robust.

12 Traditional UKF (Non-Square-Root) Formulation

The standard Unscented Kalman Filter maintains the full covariance matrix $P \in \mathbb{R}^{L \times L}$.

12.1 Covariance Prediction

The predicted covariance is calculated via a weighted outer product of the sigma point deviations:

$$P_{k|k-1} = \sum_{i=0}^{2L} W_c^{(i)} [\mathcal{X}_i - \bar{x}][\mathcal{X}_i - \bar{x}]^T + Q \tag{27}$$

12.2 Innovation and Gain

The innovation covariance S_{zz} and cross-covariance P_{xz} are computed similarly:

$$S_{zz} = \sum_{i=0}^{2L} W_c^{(i)} [\mathcal{Z}_i - \bar{z}][\mathcal{Z}_i - \bar{z}]^T + R \tag{28}$$

$$P_{xz} = \sum_{i=0}^{2L} W_c^{(i)} [\mathcal{X}_i - \bar{x}][\mathcal{Z}_i - \bar{z}]^T \tag{29}$$

The Kalman Gain K requires an explicit matrix inversion:

$$K = P_{xz} S_{zz}^{-1} \tag{30}$$

13 Implementation Details: Memory and Covariance

The comparison between the Square-Root (SR-UKF) and the Standard UKF reveals a significant difference in memory overhead and computational requirements.

13.1 Covariance Representation

The memory footprint for storing the state uncertainty differs based on the filter architecture:

- **SR-UKF Storage:** Uses the Cholesky factor $S \in \mathbb{R}^{L \times L}$. Because S is kept in a square-root form, it inherently preserves the positive-definiteness of the covariance.
- **Standard UKF Storage:** Uses the full covariance matrix $P \in \mathbb{R}^{L \times L}$.

13.2 Memory Overhead and Intermediates

The Standard UKF requires significantly more RAM on the STM32F401 because it must store large intermediate products during the Riccati update. Specifically, the following terms must be pre-allocated to avoid stack overflow:

$$P_{intermediate} = K S_{zz} K^T \tag{31}$$

While the SR-UKF performs updates in-place via Householder reflections ($O(n^2)$), the Standard UKF must maintain separate buffers for:

1. The Innovation Covariance: $S_{zz} \in \mathbb{R}^{3 \times 3}$
2. The Cross-Covariance: $P_{xz} \in \mathbb{R}^{7 \times 3}$
3. The Matrix Inverse: $S_{zz}^{-1} \in \mathbb{R}^{3 \times 3}$

13.3 Numerical Stability Limits

The Standard UKF relies on the assumption that P remains symmetric and positive-definite. On embedded hardware, floating-point rounding errors lead to:

$$P \neq P^T \implies \text{Filter Divergence} \tag{32}$$

This is the primary motivation for the SR-UKF implementation used in this project.

14 Algorithmic Comparison: Inversion vs. Back-Substitution

The fundamental performance gap between the Standard UKF and the SR-UKF on the STM32F401 originates from the method used to solve the Kalman Gain K.

14.1 Standard UKF: Matrix Inversion

The Standard UKF requires an explicit inversion of the innovation covariance $S_{zz} \in \mathbb{R}^{3\times 3}$. This is performed using the Gauss-Jordan or LU decomposition method within the CMSIS-DSP library:

$$K = P_{xz} S_{zz}^{-1} \tag{33}$$

If S_{zz} becomes ill-conditioned (nearly singular), the inversion produces massive values, leading to the "violent shaking" observed in earlier iterations.

14.2 SR-UKF: Triangular Back-Substitution

The SR-UKF avoids explicit inversion by exploiting the upper-triangular nature of the square-root innovation covariance S_z. The system solves for K in two $O(n^2)$ steps:

$$\text{Step 1: Solve } Y S_z^T = P_{xz} \quad \text{(Forward Substitution)} \tag{34}$$

$$\text{Step 2: Solve } K S_z = Y \quad \text{(Backward Substitution)} \tag{35}$$

This approach is numerically superior because it never squares the condition number of the matrix, preserving precision on 32-bit floating-point hardware.

15 Summary of Parameters for Unity Synchronization

For the 1:1 hardware-to-software mapping, the following configuration was finalized:

- **State Order:** $[x, y, z, w]$ where w is the scalar at index 3.
- **Identity Quaternion:** $[0, 0, 0, 1.0]$.
- **Measurement Gain (R_s):** 0.1 to 0.5 (Lowered to kill the 3-degree offset).
- **Process Noise (Q_s):** 0.005 (Ensures the Sigma Cloud "blooms" during motion).

15.1 The Riccati Covariance Update

In the Standard UKF, the state covariance P is updated using the Kalman Gain K and the innovation covariance S_{zz}. Unlike the SR-UKF, which updates the Cholesky factor directly, the standard version performs a full matrix subtraction:

$$P_{k|k} = P_{k|k-1} - K S_{zz} K^T \tag{36}$$

This operation is numerically sensitive; if the product $KS_{zz}K^T$ is large due to an ill-conditioned S_{zz}, the resulting P matrix may lose its positive-definiteness, eventually causing the Cholesky decomposition $P = SS^T$ to fail in the subsequent prediction step.

16 Kinematic Prediction Model

The system assumes a constant angular velocity ω over the sampling interval Δt. The state is propagated using the quaternion derivative, where the angular velocity is corrected by the estimated gyro bias b_ω:

$$\dot{q} = \frac{1}{2} q \otimes \begin{bmatrix} 0 \\ \omega - b_\omega \end{bmatrix} \tag{37}$$

For implementation on the STM32F401, we use first-order integration followed by a mandatory normalization step to maintain the unit-norm constraint:

$$q_{k+1} = \text{norm}(q_k + \dot{q}\Delta t) \tag{38}$$

This model ensures that the predicted sigma points $\mathcal{X}_{i,k+1}$ accurately reflect the rotational displacement before the measurement update occurs.

17 Higher-Order Kinematic Integration

To maintain the geometric integrity of the S^3 hypersphere, the system utilizes an *Exponential Map* (or small-angle approximation) for state propagation. Unlike first-order Euler integration, which "steps off" the unit sphere, the exponential map rotates the state along the true geodesic of the rotation manifold.

17.1 Rotation Vector and Magnitude

Given the bias-corrected angular velocity $\boldsymbol{\omega}_{corr} = \boldsymbol{\omega}_{raw} - \mathbf{b}_\omega$, the rotation magnitude θ over the interval Δt is:

$$\theta = \|\boldsymbol{\omega}_{corr}\| \Delta t \tag{39}$$

17.2 The Delta Quaternion

The incremental rotation is represented by the delta quaternion Δq. For the (x, y, z, w) state order, this is defined as:

$$\Delta q = \begin{bmatrix} \frac{\omega_x}{\|\boldsymbol{\omega}\|} \sin(\frac{\theta}{2}) \\ \frac{\omega_y}{\|\boldsymbol{\omega}\|} \sin(\frac{\theta}{2}) \\ \frac{\omega_z}{\|\boldsymbol{\omega}\|} \sin(\frac{\theta}{2}) \\ \cos(\frac{\theta}{2}) \end{bmatrix} \tag{40}$$

17.3 State Propagation (Hamiltonian Product)

The predicted state $\hat{q}_{k+1}$ is computed via the Hamiltonian product of the current state and the delta rotation. In our specific (x, y, z, w) implementation, the product $q_{new} = q_{old} \otimes \Delta q$ ensures that the local-frame gyro data correctly updates the global orientation:

$$\hat{q}_{k+1} = \begin{bmatrix} w_1x_2 + x_1w_2 + y_1z_2 - z_1y_2 \\ w_1y_2 - x_1z_2 + y_1w_2 + z_1x_2 \\ w_1z_2 + x_1y_2 - y_1x_2 + z_1w_2 \\ w_1w_2 - x_1x_2 - y_1y_2 - z_1z_2 \end{bmatrix} \tag{41}$$

18 Coordinate Frame Transformation

To align the `STEVAL-FCU1V2` hardware frame with the `Unity 3D` coordinate system, a static basis transformation is applied. The mapping ensures that hardware-local rotations correspond to the correct global Euler axes:

$$q_{unity} = \begin{bmatrix} q_x & q_z & q_y & q_w \end{bmatrix}^T \tag{42}$$

This transformation corrects the 90° offset between the `LSM6DSR` mounting orientation and the virtual drone's forward vector, resolving the Pitch-to-Yaw coupling observed during initial integration.

19 Final System Integration and Results

The implementation of the SR-UKF on the `STM32F401` successfully resolved the numerical instabilities observed in traditional UKF formulations. By utilizing a 7-dimensional state vector $[x, y, z, w, b_x, b_y, b_z]$ and the Householder-based QR decomposition, the system achieved the following performance metrics:

- **Numerical Stability:** The epsilon-guarded Householder reflection ($\epsilon = 10^{-12}$) prevented covariance collapse during stationary states.
- **Coordinate Alignment:** The $x \leftrightarrow z$ axis swap in the `Unity` converter resolved the pitch-to-yaw coupling, resulting in 1:1 orientation tracking.
- **Drift Suppression:** The tuned measurement noise ($R_s = 0.1$) suppressed gyroscopic drift to within 0.5° over a 60-second interval without a magnetometer.

19.1 Standard UKF vs. SR-UKF Observations

Comparative testing confirmed that while the Standard UKF is functional, the explicit matrix inversion (S_{zz}^{-1}) and the required frame-by-frame Cholesky refactorization ($P = SS^T$) introduced 15%−20% additional CPU overhead compared to the SR-UKF's triangular back-substitution method.

20 Matrix Storage Convention

The `ARM CMSIS-DSP` library utilizes a **Row-Major** storage format. For a matrix $M \in \mathbb{R}^{R \times C}$, the element $M_{i,j}$ is accessed at the flat index:

$$\text{index} = i \cdot C + j \tag{43}$$

Consistency between manual sigma point summation loops and the `arm_mat_mult_f32` function is critical. Any deviation results in an implicit transposition, leading to the inversion of the Kalman Gain K and subsequent filter divergence.

21 Symmetry Preservation in Standard UKF

A fundamental requirement for the Standard UKF is the maintenance of a symmetric positive-definite (SPD) covariance matrix P. Unlike the SR-UKF, which is inherently stable, the Standard UKF must explicitly enforce symmetry after the Riccati update:

$$P_k = \frac{1}{2}(P_k + P_k^T) \tag{44}$$

If P deviates from symmetry by even 10^{-7} due to rounding errors in the `arm_mat_sub_f32` function, the subsequent Cholesky decomposition for sigma point generation will fail, leading to the "Violent Shaking" observed in the Unity visualization.

22 Kalman Gain Instability in Standard UKF

During high-dynamic maneuvers, the Standard UKF implementation exhibited a Kalman Gain $K \approx 4.97$. In the context of a quaternion-based orientation filter, a gain of this magnitude leads to catastrophic over-correction:

$$\Delta q = K \cdot (z_{raw} - \hat{z}) \tag{45}$$

If $\|\Delta q\| > 1.0$, the state update results in a non-unit quaternion that violates the S^3 manifold constraints, leading to the observed "flip" phenomenon. To stabilize the baseline, the measurement noise covariance R was increased to 0.25, and an explicit symmetry enforcement loop was added to the Riccati update:

$$P_{sym} = \frac{P + P^T}{2} \tag{46}$$

This ensures that the condition number of the innovation covariance S_{zz} remains well-behaved for the `arm_mat_inverse_f32` function.

23 Quaternion Sign Ambiguity and State Flipping

The observed failure mode in the Standard UKF, where the state vector $\mathbf{q} = [0, 0, 0, 1]^T$ transitions abruptly to $\mathbf{q} \approx [1, 0, 0, 0]^T$, is a classic manifestation of the quaternion double-cover property:

$$q \equiv -q \tag{47}$$

If the `Standard_UKF_Update_Accel` calculates an innovation $\tilde{y}$ that is mathematically inverted, the Kalman Gain K generates a correction that rotates the drone 180° away from the true gravity vector. This is exacerbated in the Standard UKF due to the direct matrix inversion of S_{zz}, which can amplify small numerical asymmetries into a sign-flip in the gain matrix:

$$K = P_{xz} S_{zz}^{-1} \tag{48}$$

Numerical truncation in the 32-bit FPU leads to a singular S_{zz} during high-acceleration events, resulting in the "Violent Shaking" and subsequent state inversion.

24 Numerical State Explosion in Standard UKF

The transition of the predicted measurement $\hat{z}$ from a nominal value of 0.89 to a divergent value of $[14, -7, 4]$ indicates a failure in the *Unit Norm Constraint* of the state vector. In a quaternion-based UKF, the measurement model is defined as:

$$\hat{z} = h(q) = \mathrm{Rot}(q)\mathbf{g} \tag{49}$$

Because $\mathrm{Rot}(q)$ is an orthogonal transformation, $\|\hat{z}\|$ must equal $\|g\| \approx 1.0$. The observed values of $\|\hat{z}\| \approx 16.1$ imply that the propagated sigma points $\mathcal{X}_i$ have lost their unit magnitude. This is caused by the **Covariance Singular Point**, where the Cholesky decomposition of a non-symmetric P matrix produces "complex" sigma points that rotate the state off the S^3 manifold.

24.1 Standard Covariance Prediction and Sigma Normalization

The Standard UKF propagates the full covariance matrix $P \in \mathbb{R}^{7\times 7}$ through the nonlinear dynamics. To prevent the state explosion observed during high-dynamic maneuvers, every generated sigma point $\mathcal{X}_i$ must be explicitly re-normalized:

$$\mathcal{X}_{i,k} = \mathrm{Normalize}\left(\bar{x}_{k-1} \pm \sqrt{L+\lambda} \cdot \mathrm{chol}(P_{k-1})_i\right) \tag{50}$$

The predicted covariance is then reconstructed via the weighted outer product of the propagated deviations:

$$P_{k|k-1} = \sum_{i=0}^{2L} W_c^{(i)} (\mathcal{X}_{i,k} - \bar{x}_k)(\mathcal{X}_{i,k} - \bar{x}_k)^T + Q \tag{51}$$

where Q is the additive process noise covariance.

25 Process and Measurement Noise Parameters

For the baseline comparison, both filters utilize equivalent noise characteristics. The process noise covariance Q represents the uncertainty in the constant angular velocity model, while the measurement noise R represents the sensor variance of the `LSM6DSR` accelerometer.

$$Q = \text{diag}(\sigma_{gyro}^2, \sigma_{bias}^2) = \text{diag}(2.5 \cdot 10^{-5}, \ldots, 2.5 \cdot 10^{-5}) \tag{52}$$

$$R = \text{diag}(\sigma_{accel}^2) = \text{diag}(0.01, 0.01, 0.01) \tag{53}$$

In the SR-UKF implementation, these are stored as Cholesky factors ($Q_s = \sqrt{Q}$), whereas the Standard UKF utilizes the full 7×7 and 3×3 matrices.

26 Comparative Analysis: Drift and Smoothness

26.1 Standard UKF Numerical Deadband

While the Standard UKF implementation demonstrated high rotational smoothness, a residual drift of approximately $0.5°$ to $1.5°$ was observed during stationary tests. This is attributed to the *Numerical Deadband* inherent in the Riccati update:

$$P_k = P_{k|k-1} - K S_{zz} K^T \tag{54}$$

Due to 32-bit floating-point truncation in the `arm_mat_mult_f32` and `arm_mat_sub_f32` functions, infinitesimal innovations are occasionally lost, preventing the filter from reaching a mathematically perfect identity state.

26.2 SR-UKF Precision

In contrast, the SR-UKF maintains higher precision by operating on the Cholesky factor S using orthogonal Householder reflections. This preserves a higher dynamic range of the covariance, allowing the filter to correct smaller innovations that the Standard UKF ignores.

27 Covariance Contraction and Steady-State Error

The observed $1^\circ - 3^\circ$ static tilt in the Standard UKF, which persists even at low measurement noise ($R = 0.01$), indicates a failure in *Covariance Contraction*. In a mathematically ideal UKF, the measurement update must reduce the state uncertainty P:

$$P_{k|k} = (I - KH)P_{k|k-1} \tag{55}$$

Due to the 32-bit floating-point limitations of the `arm_mat_sub_f32` function, the term $KS_{zz}K^T$ may suffer from underflow when the innovation is small. Consequently, P remains artificially inflated, causing the filter to maintain a high "trust" in the biased gyroscope integration (Q) rather than the absolute gravity reference (R).

28 Constant Acceleration (CA) State Expansion

To support high-dynamic maneuvers and future Visual Odometry on the STM32N6, the state vector is expanded to $L = 16$ dimensions. This accounts for position, velocity, acceleration, and their respective sensor biases:

$$x = \begin{bmatrix} \mathbf{q}_{x,y,z,w} & \mathbf{b}_\omega & \mathbf{v}_{x,y,z} & \mathbf{a}_{x,y,z} & \mathbf{b}_a \end{bmatrix}^T \tag{56}$$

28.1 Kinematic Transition Logic

The prediction model now incorporates the second-order physics of motion. The velocity v and position p are updated based on the estimated acceleration a, corrected by the accelerometer bias b_a:

$$v_{k+1} = v_k + (a_k - b_a)\Delta t \tag{57}$$

$$p_{k+1} = p_k + v_k \Delta t + \frac{1}{2}(a_k - b_a)\Delta t^2 \tag{58}$$

29 Design and Tuning Parameters

Table 1: Design and Tuning Parameters for the SR-UKF (7D CV Model)

Category	**Parameter**	**Symbol**	**Value**
Scaling	Sigma Point Spread	α	0.1
Parameters	Distribution Correction	β	2.0
	Secondary Scaling	κ	0.0
Cholesky	Initial State Uncertainty	$\mathbf{S}_0$	0.1 (σ)
Factors	Process Noise (Nominal)	$\mathbf{Q}_s$	0.05 (σ_{proc})
Model	State Dimension	L	7
Complexity	QR Workspace Rows	$2L + L$	21